Shuttling Neutral Atoms Without a Zoned Layout.

Gage Erwin[1]

[1]Department of Physics, University of Wisconsin, Madison, Wisconsin, 53706, USA.

Abstract

Neutral-atom quantum processors provide dynamically reconfigurable qubit layouts through optical tweezers and acousto-optic deflectors, but this flexibility introduces a complex scheduling problem in which qubits must be physically transported to execute entangling gates while avoiding collisions, minimizing travel distance, and respecting hardware acceleration limits. Although prior work has produced efficient algorithms for initial defect-free array assembly, these methods do not address gate-level motion planning, and state-of-the-art compilers such as PowerMOVE and Enola focus on zone-based coordination strategies that report execution time and fidelity. This work introduces two unzoned shuttling compilers, naive_dag and naive_n_dag, which construct continuous, lane-based movement schedules directly from circuit DAGs and output a list of timesteps and primitives for completing the circuit. We evaluate their compilation time and execution time relative to Enola and PowerMOVE, finding that while our methods can offer lower compilation overhead, they incur longer execution times due to limited movement grouping and suboptimal path selection. Based on these comparisons, we identify concrete opportunities for improving parallelism, trajectory planning, and movement aggregation to reduce execution time and bring unzoned scheduling closer to the performance of existing zone-based approaches. We then provide an open-source project at https://github.com/Gerwinlab/NeutralMotion to facilitate further research and collaboration.

Keywords: neutral atom quantum computing (NAQC), unzone architecture (UZA),Compiler, Shuttling, QASM

1 Introduction and Motivation

Neutral-atom quantum computers have emerged as a promising platform for scalable quantum information processing. By trapping individual atoms in optical tweezers and manipulating them with acousto-optic deflectors (AODs), these systems can support large qubit arrays, dynamic rearrangement, and long-range entangling operations. Unlike fixed-connectivity architectures, neutral-atom processors can physically move qubits during computation, allowing interacting qubits to be brought together without relying only on logical SWAP networks. This reconfigurability is one of the platform's central advantages: rather than routing quantum information through many intermediate gates, the hardware can physically transport atoms so that distant qubits become adjacent or enter the same interaction region. However, this flexibility also creates a difficult compilation problem. To execute a circuit efficiently, a compiler must decide not only which gates to run and in what order, but also where each atom should be located, how atoms should move, which movements can occur in parallel, and how to avoid collisions or violations of hardware constraints during transport. Shuttling is therefore not merely a low-level implementation detail. In many circuits, movement can contribute substantially to total execution time, and poor movement schedules can erase the benefits of neutral-atom

reconfigurability. As device sizes grow, coordinating gate execution with atom transport becomes increasingly complex, making shuttling-aware compilation a central challenge for neutral-atom quantum computing.

Much prior compiler work for neutral-atom systems has focused on circuit-level optimization. These approaches reduce gate count, increase gate parallelism, or adapt logical circuits to the constraints of global or local neutral-atom control [1]. Other work goes deeper into the hardware stack by scheduling pulse-level operations in order to reduce crosstalk and shorten execution time [2]. While these methods are important, they do not fully address the cost of moving atoms during circuit execution. If dynamic shuttling is used to avoid SWAP overhead and exploit the reconfigurability of neutral-atom arrays, then movement planning must be treated as a first-class part of compilation rather than as a separate post-processing step. A separate line of work studies atom rearrangement and routing, but much of it targets initial layout preparation rather than full-circuit execution. Algorithms for defect-free array assembly, arbitrary tweezer rearrangement, and parallel atom transport are effective for preparing atoms in desired target configurations [3–6]. Similarly, optimal routing formulations can minimize transport distance and avoid collisions in atom arrays [7]. However, these methods generally solve static or single-stage reconfiguration problems. They do not directly address the repeated, multi-stage interaction between movement, gate scheduling, and qubit placement throughout an entire quantum circuit. Recent shuttling-aware compiler work begins to close this gap. For example, SMT-based layout synthesis frameworks jointly reason about placement, routing, and scheduling for dynamically reconfigurable neutral-atom hardware [8]. These formulations are valuable because they show that movement-aware compilation can be expressed as an exact constraint-solving problem. However, exact SMT-based approaches can be difficult to scale to larger circuits and devices because the search space grows rapidly with the number of qubits, gates, possible atom locations, and movement steps. In addition, the authors have not provided a public compiler implementation, making it difficult to reproduce results or directly compare against them in an experimental compiler workflow.

A related but architecturally distinct line of work comes from the trapped-ion community, particularly from the Munich group [9]. Their multi-zone ion-shuttling work studies how to coordinate ion transport across a segmented ion-trap architecture, where ions are moved between fixed storage, transport, and processing regions [10]. In this setting, shuttling is modeled using discrete hardware-specific primitives such as split, merge, shuttle, and swap operations. The Munich work also explored SAT-based formulations for encoding shuttling and scheduling constraints exactly [11]. These Boolean encodings are useful for producing feasible or optimal schedules on small instances and for demonstrating the structure of the scheduling problem. However, as with other exact methods, the SAT formulation becomes difficult to scale as the number of ions, zones, and transport operations increases. Although the Munich ion shuttler is important as an example of movement-centric compilation, it targets a fundamentally different architecture from the neutral-atom systems considered in this work. Ion shuttling occurs in segmented traps with predefined transport channels, fixed zones, and discrete shuttling primitives determined by electrode control. Neutral-atom processors, by contrast, support two-dimensional rearrangement through optical tweezers and AOD-controlled motion, allowing many atoms to move in parallel through more continuous spatial trajectories. As a result, the Munich ion shuttler provides useful conceptual motivation for scheduling physical transport, but it does not directly solve the neutral-atom problem of continuous 2D motion, parallel AOD-based shuttling, and dynamic reconfiguration during circuit execution.

Other recent systems, such as Enola [12] and PowerMOVE [13], address neutral-atom shuttling more directly. Enola introduces a scalable compiler for dynamically field-programmable neutral-atom arrays, using graph-based scheduling and placement methods to reduce the number of Rydberg stages. PowerMOVE focuses on movement optimization and reduces total shuttling time by solving a global transport problem with parallel AOD motion. These systems demonstrate that explicitly modeling movement can substantially improve execution efficiency compared with compilers that consider only logical gate structure. At the same time, these approaches rely on structured architectural assumptions, such as partitioning the device into storage, movement, and interaction zones. Zoning simplifies the scheduling problem and can reduce unnecessary motion, but it may also reduce flexibility or introduce overhead when circuit interactions do not align naturally with the chosen regions. This motivates a complementary question: how effective can

a simpler, unzoned shuttling strategy be, and where does it fall short compared with more sophisticated structured compilers?

This project investigates that question by studying continuous, unzoned shuttling for neutral-atom circuit execution. Instead of dividing the processor into predefined storage and interaction regions, we model the device as a set of movable lanes or highways and generate shuttling schedules directly from the structure of the input circuit. We introduce two baseline compilers, `naive_dag` and `naive_n_dag`, which parse QASM circuits, construct directed acyclic graphs of circuit operations, and produce physically realizable movement schedules. The `naive_dag` compiler moves one qubit at a time, while `naive_n_dag` attempts to move multiple qubits in parallel. Both compilers use greedy scheduling heuristics and are intentionally simple and transparent, making them useful baselines for understanding how much performance can be achieved from circuit-structure-driven shuttling alone. We evaluate these unzoned compilers against existing shuttling-aware approaches, including Enola and PowerMOVE, using compilation time and execution time as primary metrics. This comparison highlights the strengths of the naive methods, especially their low compilation overhead and simple implementation, while also exposing their limitations, such as redundant movement, limited batching of compatible operations, and reduced parallelism. By identifying where these baseline approaches fail, we motivate future compiler designs that combine the flexibility of unzoned motion with more global movement optimization.

The remainder of this report is organized as follows. Section 2 describes the model and methods used by the proposed greedy shuttling solvers, including the highway abstraction, DAG-based circuit representation, movement scheduling procedure, and configuration parameters that determine hardware and routing behavior. Section 3 presents the experimental results, including simulated-annealing experiments and comparisons against Enola and PowerMOVE on benchmark circuits. Section 4 discusses the full-stack impact of automatically translating logical circuits into hardware-level movement and gate schedules. The final sections summarize the limitations of the current implementation, outline future improvements such as better cost functions and more flexible AOD movement models, and provide information about code availability and installation.

2 Model and Methods

We model the neutral-atom processor as a two-dimensional array of trapping sites. Each site may either contain a neutral-atom qubit or remain vacant, with vacant sites providing the free space needed for shuttling during circuit execution. Unlike a fixed-connectivity architecture, this model allows atoms to be physically transported so that qubits required for a two-qubit gate can be brought within the Rydberg interaction distance. To make this movement problem tractable, we create the trapping array with structured movement pathways, which we refer to as *highways*. Highways define the allowed lanes along which atoms may be transported, and SLM traps are not placed. Rather than permitting arbitrary continuous motion across the full two-dimensional plane, the solver restricts motion to horizontal and vertical highway paths. This abstraction simplifies routing, reduces the number of possible movement choices, and helps the scheduler generate collision-free movement schedules. Occupied sites act as obstacles that must be avoided, while vacant sites provide intermediate positions that can be used during transport. Figure 1 illustrates this model. In the example, qubit Q_0 remains fixed while qubit Q_1 is moved along the highway network toward an interaction location. Once the moving atom reaches a neighboring highway position within the prescribed interaction radius, the corresponding two-qubit gate can be executed. The highway abstraction also helps enforce spatial separation between simultaneous operations. By maintaining a minimum spacing between neighboring movement lanes and interaction regions, the model reduces the risk that atoms participating in two-qubit gates unintentionally interact with atoms involved in another gate. This is important because neutral-atom entangling gates are mediated by Rydberg interactions, so unintended proximity between atoms can lead to crosstalk or unwanted coupling during excitation.

For computational simplicity, the solver discretizes the device into a lattice with separate roles for storage and movement positions. Fixed neutral atoms are placed on even-even row-column coordinates. Moving atoms occupy adjacent highway positions, represented by even-odd or odd-even coordinates. Under

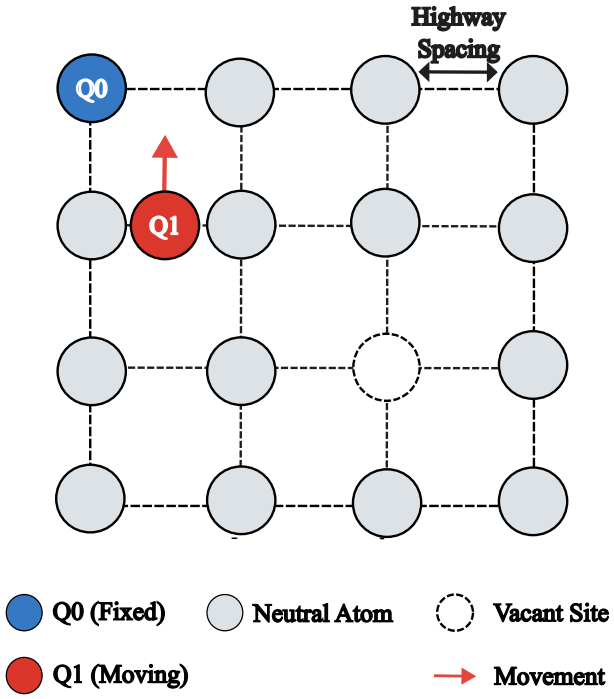


Fig. 1: Highway-based movement model for neutral-atom shuttling. The processor is represented as a two-dimensional array of trapping sites connected by horizontal and vertical movement highways. Occupied sites contain neutral atoms, while vacant sites provide available space for routing. In this example, qubit Q_0 remains fixed and qubit Q_1 is transported along the highway network until it reaches an interaction position near Q_0. The highway spacing parameter controls the physical separation between neighboring movement lanes and helps maintain isolation between simultaneous two-qubit gate operations.

this discretization, two atoms are considered to be within interaction range when the moving atom occupies a lattice site adjacent to the fixed atom, such as an offset of $(\pm 1, 0)$ or $(0, \pm 1)$ from the fixed atom's position. This discrete representation allows the compiler to reason about shuttling using simple grid operations while still capturing the physical requirement that atoms must be brought close enough for a Rydberg-mediated two-qubit gate. Before compiling the physical spacing for the highways can be specified, which determines the separation between adjacent lattice positions. This allows the same compiler framework to match different hardware assumptions or benchmark geometries. For most benchmarks in this report, we set the highway spacing to 15 μm, corresponding to the Rydberg interaction radius assumed by the scheduler. For the $[[144, 12, 12]]$ bivariate bicycle code benchmark, we instead use a spacing of 5 μm to remain consistent with the layout assumptions of the corresponding prior work [14]. This configuration enables comparison between general neutral-atom benchmark circuits and the specialized geometry used for the bivariate bicycle code experiment.

The physical layout is specified by two configurable grids. The first is an $n \times m$ lattice of trapping sites, which must be large enough to contain all atoms required by the input circuit. This lattice represents the static SLM-defined array used for atom storage and gate locations. The second is a $k \times l$ AOD grid, which defines the set of mobile traps available for shuttling atoms through the plane. In most experiments, the AOD grid is chosen to have the same dimensions as the lattice, although the two grids are specified independently in the configuration file. A two-dimensional AOD generates mobile traps at the intersections of controllable rows and columns. Atoms loaded from the SLM into the AOD can then be transported by moving these

rows and columns. The current solver makes two simplifying assumptions about this process. First, atoms loaded into the AOD preserve their relative row and column ordering, meaning that AOD rows and columns are not allowed to cross during transport. This reflects the fact that AOD-controlled traps move as ordered sets of rows and columns rather than as fully independent point particles. Second, the solver does not allow the AOD grid to expand or stretch during execution. This restriction reduces the search space and provides a simpler baseline model for movement scheduling. Under this assumption, a set of atoms can be loaded into the AOD only when the configured AOD grid is large enough to accommodate them.

The hardware and timing assumptions used by the solver are specified through a JSON configuration file. This file defines both architectural parameters, such as the lattice size and AOD grid size, and timing parameters that determine the duration of gates, loading operations, pulse switching, and movement. The required timing keys include `average_single_gate_time`, which specifies the duration of a single-qubit gate; `average_two_gate_time`, which specifies the duration of a two-qubit gate; `t_switch`, which captures the time required to switch or retarget the control pulse between operations; and `transfer_SLM_AOD`, which gives the time required to transfer atoms between the static SLM traps and the mobile AOD traps. The configuration file also specifies `max_acceleration` and `max_velocity`, which bound the allowed AOD motion. These parameters are intended to be hardware-specific inputs and can be tuned to reflect device constraints, including limits chosen to reduce motional heating and preserve atom-trap fidelity during shuttling. Given these parameters, the current solvers estimate movement duration using a trapezoidal velocity profile. Under this model, an atom accelerates up to the configured maximum velocity, moves at constant velocity when the transport distance is long enough, and then decelerates to rest at the target location. For shorter movements where the maximum velocity cannot be reached, the profile reduces to a triangular accelerate-decelerate trajectory. This timing model provides a simple and physically interpretable baseline for estimating AOD transport time. The compiler framework is extensible, however, and future versions could incorporate additional motion profiles, such as smoother jerk-limited trajectories or experimentally calibrated movement curves, to better match hardware-specific transport behavior.

2.1 Directed Acyclic Graph

Once the hardware configuration has been fully specified, the solver takes as input a QASM file encoding the quantum circuit as a sequence of gate primitives in OpenQASM 3.0. We use Qiskit to parse this circuit and convert it into a directed acyclic graph (DAG), where nodes represent gate operations and edges represent data dependencies between gates. This representation allows the scheduler to identify sets of gates that can be executed in parallel, as shown in Figure 2. In particular, gates appearing at the same depth of the DAG are independent of one another and can be scheduled in the same execution stage without violating circuit dependencies. Before constructing the two-qubit shuttling schedule, the solver separates single-qubit gates from the main movement-planning problem. Since single-qubit gates do not require atoms to be transported into an interaction region, they are removed from the circuit used to generate the two-qubit DAG. However, they are not discarded. Instead, the solver records them in an ordered array that preserves their dependency relationship with the surrounding two-qubit gates. Specifically, the gates stored at index i of this array are scheduled to occur before the ith stage of two-qubit gates. This allows the compiler to simplify the shuttling problem while still ensuring that all single-qubit operations are executed at the correct point in the logical circuit. This DAG-based staging approach differs from compilers such as Enola and PowerMOVE [12, 13], which use graph-theoretic methods such as edge coloring to exploit commutation structure and reduce the number of interaction stages. In contrast, our solver uses the DAG primarily as a simple dependency-preserving schedule: it does not attempt to globally reorder commuting gates or optimize the number of stages beyond the dependency structure exposed by the circuit DAG. The computational cost of constructing and processing the DAG is dominated by the Qiskit conversion step. For a circuit with g gates and n qubits, this step scales approximately linearly in the number of gates, with time complexity $O(g)$, while the memory requirement scales with the size of the dependency representation. In practice, the dominant challenge is therefore not constructing the logical schedule, but mapping each scheduled two-qubit interaction

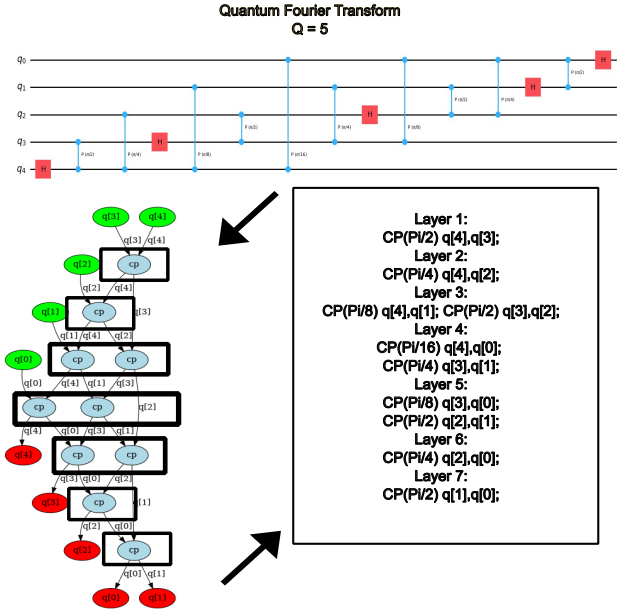


Fig. 2: Conversion of an input OpenQASM circuit of QFT with 5 qubits into a directed acyclic graph (DAG) representation. Gates become DAG nodes, dependency constraints become edges, and nodes at the same DAG depth form execution stages that can be scheduled in parallel without violating circuit dependencies.

onto physical atom positions while minimizing movement overhead and satisfying the highway, AOD, and collision constraints described above.

2.2 Initial Layout Selection

Once the circuit has been decomposed into single-qubit gate layers and two-qubit gate stages, the scheduler must choose an initial physical placement for the logical qubits. This initial layout determines where atoms begin on the neutral-atom lattice and can have a significant impact on the total amount of shuttling required during execution. A poor initial placement may force frequently interacting qubits to travel long distances, while a better placement can reduce movement overhead by placing qubits with repeated interactions closer together. In our framework, the initial layout strategy is specified in the configuration file. We support three layout initialization modes: `random`, `given`, and `fast_sa`.

The simplest option is `random` initialization. In this mode, the compiler randomly assigns logical qubits to valid sites on the lattice. This strategy is useful as a baseline because it makes no attempt to exploit circuit structure. However, because it ignores the interaction pattern of the circuit, it can produce layouts with large communication distances between frequently interacting qubits.

The second option is `given` initialization. In this mode, the configuration file provides an explicit array of atom placements, and the compiler uses this placement directly. This allows the user to test hand-designed layouts, layouts produced by an external tool, or layouts taken from a previous experiment. The `given` mode is particularly useful when comparing against prior work or when the benchmark circuit has a known geometry, such as the $[[144, 12, 12]]$ bivariate bicycle code layout. Since the placement is externally specified, the solver does not modify the initial assignment before scheduling begins.

The third option is `fast_sa`, which uses fast simulated annealing to search for a better initial placement before shuttling begins [15]. This method attempts to minimize a layout cost function that rewards short

interaction distances and favorable alignment of movements within each stage. Let $\mathcal{L}$ denote the set of two-qubit gate stages, and let v_g denote the movement vector associated with gate g in a stage l. The layout cost is defined as

$$C = \sum_{l \in \mathcal{L}} \left(\sum_{v_g \in l} \frac{d_{l,g}}{N_l(v_{l,g})} - \sum_{g,g' \in l} \cos(v_g, v_{g'}) \frac{\min(\|v_g\|, \|v_{g'}\|)}{\max(\|v_g\|, \|v_{g'}\|)} \right).$$

Here, $d_{l,g}$ is the distance between the atoms participating in gate g during stage l, and $N_l(v_{l,g})$ is the number of identical movement vectors appearing in the same stage. The first term penalizes long-distance interactions, encouraging qubits that interact to be placed closer together. Dividing by $N_l(v_{l,g})$ reduces the penalty for repeated identical movement vectors, which encourages layouts that create reusable or groupable movement patterns. The second term rewards movement vectors that are aligned in direction and similar in magnitude. This favors layouts where multiple atoms can be moved in parallel using AOD motions.

During simulated annealing, the solver repeatedly proposes small changes to the layout, such as swapping the positions of two atoms. Each proposed layout has an associated change in cost, ΔC. If the new layout improves the cost, the move is accepted. If the new layout is worse, it may still be accepted with probability

$$P_{\text{accept}} = \min\left(1, \exp\left(\frac{-\Delta C}{T_n}\right)\right),$$

where T_n is the temperature at annealing step n. This probabilistic acceptance rule allows the search to escape poor local minima early in the optimization process. As the temperature decreases, the algorithm becomes increasingly greedy and eventually accepts mostly cost-improving moves. The temperature schedule is defined as

$$T_n = \begin{cases} \frac{\Delta_{\text{avg}}}{\ln \frac{1}{P}}, & n = 1, \\ \frac{T_1 \langle \Delta_{\text{cost}} \rangle}{nc}, & 2 \le n \le k, \\ \frac{T_1 \langle \Delta_{\text{cost}} \rangle}{n}, & n > k, \end{cases}$$

where Δ_{avg} is the average uphill cost, $\langle \Delta_{\text{cost}} \rangle$ is the average observed cost change, c controls the early cooling rate, P is the initial probability to accept uphill solutions, and k determines when the schedule transitions to the later cooling regime.

From the temperature, you can see that the annealing procedure is divided into three stages, each of which uses a different type of layout update. The first stage, corresponding to $n = 1$, is fully random. In this stage, proposed layouts are accepted unconditionally, regardless of whether they improve or worsen the cost. This provides an initial exploration phase and prevents the search from being biased too strongly by the starting placement. The second stage uses greedy, structure-aware moves. In this phase, the solver selects a random two-qubit gate stage from the circuit and modifies the placement of the qubits involved in that stage. The goal is to move those qubits into randomly chosen valid locations that increase the number of compatible parallel movements. A proposed move is accepted only if it improves the layout cost. This stage therefore acts as a greedy refinement step, using the circuit's staged interaction structure to encourage placements that produce more parallelizable shuttling patterns. The final stage performs broader random perturbations of the layout. Instead of focusing only on qubits from a single selected gate stage, the solver randomizes the positions of a chosen number of atoms across the grid. These larger moves allow the search to escape layouts that may have been overfit to earlier local refinements. Proposed layouts in this stage are evaluated using the same cost function, with acceptance determined by the annealing temperature schedule. Together, these three stages provide a balance between global exploration, greedy improvement of stage-level parallelism, and broader randomized layout refinement.

The cost of one `fast_sa` evaluation is dominated by the largest two-qubit gate stage. If $s_{\max}$ denotes the number of gates in the largest two-qubit stage, then the distance term requires evaluating approximately $O(s_{\max})$ gate vectors per stage, while the alignment term compares pairs of movement vectors within a stage and therefore scales as $O(s_{\max}^2)$. Thus, a single cost-function evaluation is roughly $O(Ls_{\max}^2)$ in the worst

case, where L is the number of stages. If the annealing procedure performs N_{SA} proposed updates, the total optimization overhead is approximately $O(N_{\mathrm{SA}} L s_{\max}^2)$, in addition to the cost of constructing the staged circuit representation. In the results section, we use these initialization strategies to study how much initial placement quality affects the final shuttling schedule and total execution time.

2.3 Naive_Dag

The `Naive_Dag` solver was developed as a simple baseline for evaluating the benefits and limitations of more parallel shuttling strategies. Unlike `Naive_N_Dag`, which attempts to move multiple atoms at the same time, `Naive_Dag` moves only one atom at a time. This makes the resulting schedules easy to interpret and provides a useful benchmark for identifying which performance improvements come from parallel AOD motion, movement grouping, or more sophisticated routing decisions. The solver operates on a one-dimensional dependency ordering derived from the circuit DAG. For each two-qubit interaction, one of the participating qubits is selected and loaded from its static SLM trap into the AOD. The loaded atom is then moved along the highway network until it is within interaction range of the other qubit, which remains fixed in its SLM position. After the two atoms are brought within the required Rydberg radius, the corresponding two-qubit gate is executed. Although `Naive_Dag` moves only one atom at a time, it is designed to keep the loaded atom in the AOD for as long as possible while preserving circuit dependencies. If the atom currently in the AOD has another dependency-ready two-qubit gate, the solver keeps that atom on the highway rather than immediately returning it to its original SLM trap. In this case, the atom is routed to the next interaction partner and executes the next available gate. Because the atom remains mobile, each subsequent interaction is scheduled so that the moving qubit can be brought within range of the fixed partner using at most one turn in the highway network. This reduces unnecessary SLM-AOD transfers and allows a single loaded atom to execute a sequence of valid interactions before being unloaded. Once the loaded atom has no remaining dependency-ready interactions, it is returned to its corresponding SLM trap and transferred out of the AOD. The solver then selects the next qubit with an available two-qubit interaction, loads it into the AOD, and repeats the process. This produces a sequential movement schedule in which each mobile episode consists of loading one atom, performing as many valid interactions as possible while that atom remains in the AOD, and then unloading it back to the SLM. Single-qubit gates are handled separately from the shuttling process. Since they do not require atom transport, the solver schedules them only during periods when no atoms are moving. This avoids overlap between single-qubit control pulses and AOD transport, simplifying the timing model and ensuring that all single-qubit operations are inserted at dependency-preserving points in the execution schedule. Overall, `Naive_Dag` provides a deliberately conservative baseline: it avoids the complexity of simultaneous atom motion while still exploiting the ability to keep one atom mobile across multiple dependent interactions.

*Note - The current implementation of Naive_Dag only supports random initial layout.

2.4 `Naive_N_Dag`

The `Naive_N_Dag` solver extends the `Naive_Dag` baseline by allowing multiple atoms to move during the same execution stage. Rather than shuttling a single atom through all of its available interactions, `Naive_N_Dag` processes the two-qubit DAG stage by stage and attempts to group compatible movements so that several gates can be prepared in parallel. The solver is still heuristic and greedy, but it provides a more flexible model for studying how parallel AOD motion affects execution time, unnecessary movement, and scheduling fidelity. For each two-qubit stage, the solver first computes a movement vector for each gate. This vector represents the displacement required to bring the selected moving atom from its current position to an interaction position near its fixed partner. The solver then groups gates according to the alignment of their movement vectors. For a candidate group of gates in a stage l, the alignment score is computed using

the pairwise vector-alignment term

$$A_l = \sum_{g,g' \in l} \cos(v_g, v_{g'}) \frac{\min(\|v_g\|, \|v_{g'}\|)}{\max(\|v_g\|, \|v_{g'}\|)},$$

where v_g and $v_{g'}$ are movement vectors for gates g and g'. This score rewards movements that point in similar directions and have similar magnitudes. Intuitively, gates with highly aligned vectors are easier to execute together because their atoms can be moved using compatible AOD row and column motion. The user specifies an alignment threshold in the configuration file, with values between -1 and 1. This threshold controls how aggressively the solver groups movements. A threshold of -1 allows all gates in a stage to be grouped together regardless of movement direction, maximizing parallelism but potentially introducing unnecessary or poorly aligned motion. A threshold of 1 is the most restrictive setting and only groups gates whose movement vectors are parallel and highly compatible. Intermediate values allow the user to tune the tradeoff between movement parallelism and movement efficiency. This parameter is useful for studying how additional motion affects total execution time and may also be useful for future fidelity studies where excess shuttling or poorly aligned transport could increase error.

The grouping procedure is greedy. The solver iterates through the gates in a stage and attempts to assign each gate to an existing group. For each candidate gate, the solver evaluates which choice of moving atom best fits the current group according to the alignment score. If either possible moving atom produces a movement vector that satisfies the alignment threshold for the group, the best-scoring option is added to that group. If neither atom produces a compatible vector, the solver either attempts to place the gate into another existing group or creates a new group. In this way, each stage is partitioned into one or more movement groups, where each group contains gates that can be prepared using sufficiently compatible transport directions. After the groups have been created, the gates within each group are ordered according to their alignment and movement distance. Gates with shorter or more representative movement vectors are prioritized first, while later gates are selected to follow the dominant direction of the group whenever possible. This ordering encourages atoms to move in directions that serve multiple gates and reduces unnecessary detours. Because the groups are defined in terms of movement vectors, the solver updates vectors relative to the atom's current position after each move. Thus, the first vector points from the atom's SLM position to the location needed for the first gate, while the second vector points from the atom's position after the first gate to the location needed for the second gate. This relative-vector update allows the solver to reason about sequential movement within a group rather than treating every gate as an independent move from the original SLM position. The solver also performs a limited look-ahead step to reduce stop-and-go motion. Before committing to a movement, it checks whether the next gate in the group can be reached with a nearby or compatible displacement. If a single movement can place the atom close enough to execute both the current and next gate, the solver chooses that movement instead of stopping after the first interaction and accelerating again for the next one. This behavior is intended to reduce unnecessary acceleration and deceleration cycles, which can add substantial execution time under the trapezoidal motion model. Figure 3 illustrates this multi-gate movement behavior. Once all atoms in a group have reached their required interaction positions, the corresponding two-qubit gates are executed in parallel. After a group finishes, the solver checks the next group in the following DAG stage. If the next group is identical to the current group, the atoms remain in the highway and continue directly to the next scheduled interactions. Otherwise, the atoms are unloaded from the AOD and returned to their SLM traps before the next group is loaded. This conservative unload policy simplifies dependency management and avoids carrying atoms through unrelated stages, but it may introduce additional SLM-AOD transfer overhead.

Overall, `Naive_N_Dag` is a greedy heuristic solver for parallel neutral-atom shuttling. It does not globally optimize the full movement schedule, nor does it perform exhaustive search over all possible atom assignments, groupings, or trajectories. Instead, it uses local alignment-based decisions to create parallel movement groups within each DAG stage. This makes the solver useful as a transparent baseline for studying how much performance can be gained from simple parallel shuttling and where more sophisticated global optimization would be needed.

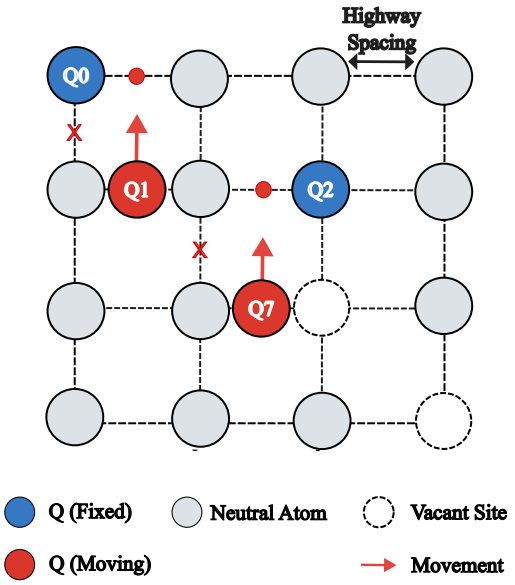


Fig. 3: Multi-atom shuttling behavior in `Naive_N_Dag`. Gates within a DAG stage are grouped according to the alignment of their movement vectors. Here we depict two qubits moving together in the AOD where the solver moves them to a position so both atoms can have a gate done simultaneously.

3 Results

3.1 Output Schedule Format

After the movement and gate stages have been generated, the solver emits a time-ordered schedule describing the physical actions required to execute the circuit. This output is intended to make the compiled program readable while still exposing the hardware-level operations needed for neutral-atom execution. Each line of the schedule corresponds to either an initialization instruction, an SLM-AOD transfer, an atom movement, or a quantum gate. The `initialize` instruction specifies the fixed home position of each atom in the lattice. These positions define where atoms begin execution and where they return after being transported through the AOD. A `load` instruction indicates that an atom is transferred from its static Spatial Light Modulator (SLM) trap into the mobile two-dimensional Acousto-Optic Deflector (AOD) transport system. In the scheduler's timing model, the duration of a `load` operation includes both the configured SLM-to-AOD transfer time and the motion required for the atom to enter the transport highway. The `unload` instruction represents the reverse process: the atom leaves the AOD transport path and returns to its fixed SLM lattice site. Movement instructions specify the atom being transported and its displacement across the lattice. For example,

```
move q[5] (2,1) : (0,1)
```

indicates that atom `q[5]` moves from lattice coordinate $(2, 1)$ to coordinate $(0, 1)$. The geometric displacement can be inferred from these coordinates, while the scheduler computes the corresponding movement duration internally using the configured `lattice_spacing`, `max_velocity`, `max_acceleration`, and trapezoidal movement profile. Gate instructions are emitted in an OpenQASM 3.0-style textual form. For example, single-qubit gates may appear as `h q[5];`, while two-qubit gates may appear as `cp(pi/2) q[5],q[4];`. These gate lines are interleaved with load, move, and unload operations to form a complete execution timeline. This explicit schedule format makes it possible to inspect how the compiler maps a logical circuit into physical atom transport and gate execution.

3.2 Simulated Annealing

To evaluate the effect of initial placement optimization, we applied the `fast_sa` initialization method to the $[[144, 12, 12]]$ bivariate bicycle code benchmark. This benchmark was chosen because prior work shows that a full syndrome-extraction cycle for this code can be executed in only a small number of movement rounds when the atoms are placed in a carefully structured initial layout, producing a reported execution time of approximately 2.97 ms [14]. This makes the benchmark especially useful for testing whether our placement heuristic can discover layouts that expose similar parallel movement structure.

For this experiment, we used a 17×17 lattice and configured the hardware parameters to match the assumptions used for the bivariate bicycle comparison. The SLM-AOD transfer time was set to 15 μs, the maximum AOD acceleration to 2750 m/s^2, and the maximum AOD velocity to 0.55 m/s. The Rydberg radius and lattice spacing were set to 5 μm. We used an average single-qubit gate time of 0.1 μs, an average two-qubit gate time of 0.27 μs, and a pulse switching time of 1.5 μs. Movement durations were computed using the trapezoidal movement profile described in Section 2. Figure 4 shows the behavior of the greedy

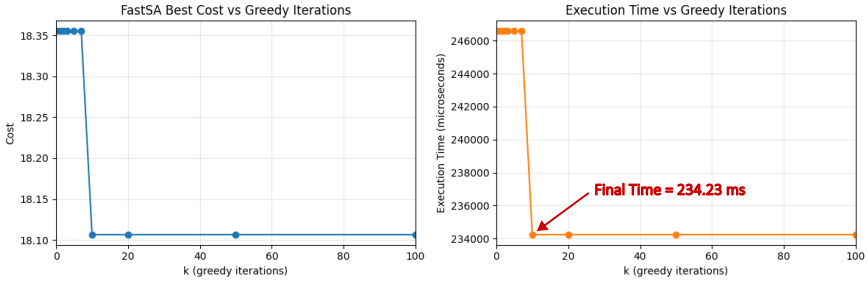


Fig. 4: Effect of greedy `fast_sa` placement updates on the $[[144, 12, 12]]$ bivariate bicycle code benchmark. The left plot shows the placement cost decreasing rapidly during the first few greedy iterations, followed by a long plateau. The right plot shows the corresponding execution time, which improves from approximately 240 ms to 215.228 ms but remains far above the execution time obtained from a carefully structured initial placement.

stage of the `fast_sa` optimizer. In this experiment, we only step through the greedy update stage in order to isolate how much improvement can be obtained from local placement refinements. The placement cost decreases quickly at the beginning of the optimization, dropping from approximately 18.35 to 18.1 after only a few iterations. However, after this initial improvement, the cost reaches a plateau and remains nearly unchanged for thousands of additional iterations. This indicates that the greedy update rule can find simple local improvements, but it struggles to escape local minima or discover the highly structured layout required by this benchmark.

The execution-time trend shows a similar limitation. The initial schedule takes roughly 248 ms, and the greedy placement updates reduce this to 234.23 ms. Although this is a measurable improvement, the resulting execution time is still very poor compared with the few-millisecond schedule reported for the same code under a specialized layout [14]. To test whether additional optimization could overcome this limitation, we repeated the experiment with a longer run consisting of 2000 greedy iterations and 100000 third-stage randomization iterations. The best execution time found in this longer search was 206.48 ms, only a minor improvement over the shorter greedy run.

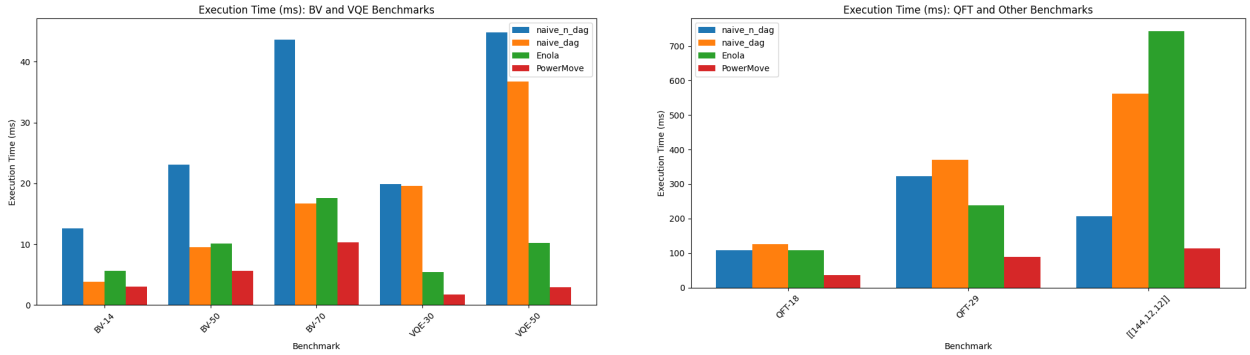
We also evaluated the scheduler using a given initial placement based on the layout from Ref. [14]. With this externally supplied placement, the same scheduler achieved an execution time of 19.025 ms. This is still slower than the reported 2.97 ms result, but it is an order of magnitude faster than the layouts found by our current simulated-annealing procedure. These results suggest that the poor performance is not caused solely by the shuttling scheduler. Instead, the initial placement has a dominant effect on execution time for this benchmark, and the current `fast_sa` placement heuristic is not yet strong enough to recover the highly

structured layout needed for efficient bivariate bicycle code execution. Future work should therefore improve the placement cost function, and introduce larger and more structured layout moves.

3.3 Comparison to PowerMOVE and Enola

In this section, we evaluate the full scheduler against Enola and PowerMOVE. For the $[[144, 12, 12]]$ bivariate bicycle code benchmark, we use the same hardware configuration described in the simulated annealing section. For all other benchmarks, we use the hardware parameters from PowerMOVE [13]: an SLM-AOD transfer time of 15 μs, maximum AOD acceleration of 2750 m/s^2, maximum AOD velocity of 0.55 m/s, Rydberg radius of 15 μm, average single-qubit gate time of 1 μs, average two-qubit gate time of 0.27 μs, pulse switching time of 1.5 μs, and all atoms in the same stage are moved together.

We evaluate four benchmark families. The Bernstein-Vazirani benchmarks use circuits with 14, 50, and 70 qubits. The VQE benchmarks use 30- and 50-qubit circuits, with QASM files taken from PowerMOVE [13]. We also evaluate Quantum Fourier Transform circuits with 18 and 29 qubits, as well as the $[[144, 12, 12]]$ bivariate bicycle code. These benchmark families represent different interaction structures. QFT and the bivariate bicycle code have more regular structure and expose many opportunities for parallel gate execution. VQE circuits are less regular and contain more varied interaction patterns. BV circuits are not highly structured for broad parallel gate execution, but they repeatedly interact with a small set of qubits, making them useful for testing whether a scheduler can exploit repeated use of the same atoms. We present the data in a bar graph presented in Figure 5 and have the exact times along with compilation time in Table 1.



(a) BV and VQE benchmark comparison.

(b) QFT and $[[144, 12, 12]]$ benchmark comparison.

Fig. 5: Execution-time comparison across benchmark families for Enola, PowerMOVE, `Naive_Dag`, and `Naive_N_Dag`. The BV and VQE benchmarks test less regular or hub-like interaction patterns, while the QFT and $[[144, 12, 12]]$ benchmarks contain more structured interaction patterns with greater potential for parallel execution.

Benchmark	Enola T_{exe} (μs)	PowerMOVE T_{exe} (μs)	Naive_Dag T_{exe} (μs)	Naive_N_Dag T_{exe} (μs)	Enola T_{comp} (s)	PowerMOVE T_{comp} (s)	Naive_Dag T_{comp} (s)	Naive_N_Dag T_{comp} (s)
BV-14	5583.98	3034.20	3785.15	12648.30	669.48	28.79	0.37	81.58
BV-50	10118.96	5631.26	9504.69	23122.36	1710.91	17.95	0.38	105.11
BV-70	17620.11	10277.27	16670.12	43578.97	4334.50	20.30	0.41	199.59
QFT-18	108173.62	36810.15	125422.97	108267.12	10917.80	347.47	0.47	343.78
QFT-29	239150.00	89670.26	371220.53	323181.40	24116.00	511.97	0.64	1002.54
VQE-30	5436.18	1688.03	19578.20	19897.12	57.62	29.68	0.43	104.52
VQE-50	10196.50	2946.26	36699.64	44832.21	56.58	76.01	0.42	232.10
[[144, 12, 12]]	742617.95	113226.17	562868.67	206748.66	258.15	30.36	2.14	1898.63

Table 1: Comparison of execution time and compilation time across four compilation strategies: Enola, PowerMOVE, `Naive_Dag`, and `Naive_N_Dag`. Execution time is reported in microseconds and reflects the end-to-end scheduled runtime on the neutral-atom architecture. Compilation time is reported in seconds and captures the classical overhead required to generate the full movement-and-gate schedule.

From Figure 5, we observe several trends across the benchmark families. In the BV benchmarks shown in Figure 5(a), `Naive_N_Dag` performs poorly relative to the other methods, while `Naive_Dag` slightly outperforms Enola. The poor performance of `Naive_N_Dag` suggests that its attempted parallelism is outweighed by repeated AOD loading and unloading overhead. Although some parallel gates may be available, the solver frequently returns atoms to the SLM and reloads them in later stages, accumulating transfer and movement costs that dominate any benefit from parallel execution. By contrast, the performance of `Naive_Dag` on the BV benchmarks is encouraging. Since BV circuits repeatedly interact with a small number of qubits, the single-atom strategy can keep one useful atom in the AOD across multiple interactions and avoid unnecessary transfers. This result suggests that an unzoned layout can be competitive when the circuit structure contains repeated interactions involving the same atoms. It also indicates a possible path for improvement: the current benchmarks use a 15 μm lattice spacing, and reducing this spacing to 5 μm would decrease movement distances and could substantially reduce execution time. If this reduction were combined with more effective parallel-gate scheduling, an improved unzoned solver may be able to approach or exceed PowerMOVE on BV-style circuits. The VQE benchmarks show a different behavior. In these circuits, neither `Naive_Dag` nor `Naive_N_Dag` performs competitively with Enola or PowerMOVE. This suggests that VQE circuits do not provide the same repeated-qubit structure that `Naive_Dag` exploits in BV. Instead, atoms are repeatedly switched in and out of the AOD, and the solver is unable to maintain useful atoms in the highway across multiple interactions. At the same time, `Naive_N_Dag` does not find enough compatible parallel movements to compensate for the additional transfer overhead. As a result, both naive solvers suffer from excessive shuttling and limited useful parallelism.

Figure 5(b) shows that more structured circuits can benefit from multi-atom shuttling. In the QFT benchmarks, `Naive_N_Dag` outperforms `Naive_Dag`, indicating that the alignment-based grouping heuristic is able to exploit some of the parallel structure in the circuit. However, `Naive_N_Dag` still does not outperform Enola or PowerMOVE. A major limitation is the current unload policy: even when nearly the same set of atoms is used in the next stage, adding or removing a single atom causes the entire AOD group to be returned to the SLM and reloaded. This introduces unnecessary transfer overhead and prevents the solver from preserving useful partial groups across stages. The $[[144, 12, 12]]$ bivariate bicycle benchmark further illustrates the importance of placement and structured movement. After many `fast_sa` iterations, `Naive_N_Dag` outperforms both `Naive_Dag` and Enola on this benchmark, showing that the multi-atom strategy can be effective when the circuit has highly structured parallel interactions. Nevertheless, it remains slower than PowerMOVE, even though this benchmark uses a smaller 5 μm lattice spacing. Together with the simulated annealing results, this suggests that initial placement has a dominant effect on execution time, and that the current placement heuristic is still not strong enough to reliably recover the highly optimized geometry required for this code. This is supported by the given atom placement in the Simulate Annealing Section that resulted in 19 ms of execution time which greatly outperforms PowerMove.

Table 1 also highlights a clear compilation-time tradeoff. `Naive_Dag` has by far the lowest compilation time because it uses a very simple sequential scheduling strategy and performs little optimization. In contrast, `Naive_N_Dag` often requires much longer compilation time because each benchmark is run with many `fast_sa` iterations in an attempt to improve the initial placement and reduce execution time. Although additional annealing iterations could potentially produce better schedules, the results in the previous section indicate that the current `fast_sa` procedure is not sufficient by itself. Both Enola and PowerMOVE also use simulated annealing, but their placement problem is simpler because their zone-based architectures primarily optimize the distance from memory to interaction regions. In our unzoned solver, the cost function must also account for two-dimensional AOD movement constraints inside the interaction region, making the placement and scheduling problem more difficult.

4 Full-Stack Impact / Resource Implications

The primary impact of this work lies in bridging the gap between the logical quantum program and the physical execution of neutral-atom hardware. In current neutral-atom systems, translating a high-level

quantum circuit into executable instructions is a complex and often manual process, requiring careful coordination of atom movement, gate timing, and hardware constraints. This scheduler removes that burden by automatically generating a complete sequence of hardware-level instructions from a logical circuit description. Instead of manually constructing movement schedules and gate timelines, the system produces a fully specified execution plan, significantly reducing the effort required to program and operate neutral-atom devices. At the physical layer, the system directly impacts how neutral atoms are controlled and manipulated. The scheduler outputs explicit movement and gate instructions that can be consumed by a hardware controller, ensuring that all required atom movements and interactions are correctly ordered and timed. This eliminates the need for hand-crafted control sequences and reduces the likelihood of human error in coordinating complex motion patterns. As circuit size increases, the difficulty of manually designing such sequences grows rapidly, making automation essential for scalability. From a systems perspective, this work establishes a clearer interface between the logical and physical layers of the quantum computing stack. By providing a deterministic and automated translation from circuits to control instructions, it enables a more standardized workflow in which high-level programs can be compiled directly into executable schedules. This not only improves developer productivity but also facilitates integration with control software and hardware drivers, as the output format can serve as a consistent input to lower-level execution systems. In addition to improving usability, this automation has important resource implications. Generating instructions programmatically ensures more consistent and efficient use of hardware resources, as movement and gate operations are coordinated systematically rather than heuristically by hand. It also reduces development time and lowers the barrier to experimenting with larger and more complex circuits.

5 Limitations and Future Work

Overall, the current solver succeeds in taking an input QASM circuit and producing a valid neutral-atom execution schedule. It automatically generates initialization, SLM-AOD transfer, movement, gate, and unload instructions while respecting the simplified highway and AOD constraints described above. However, this is still only a first step toward a full-stack neutral-atom compiler. A complete implementation must support more than a static circuit description, it should also support fault-tolerant execution patterns such as repeated syndrome extraction, ancilla measurement, and fault-tolerant ancilla preparation. These requirements introduce additional scheduling constraints because ancilla qubits may be measured, reset, and reused across many cycles, while data qubits must remain protected from unnecessary movement and decoherence. Supporting this broader workflow will require expanding the compiler's OpenQASM 3.0 functionality. Since OpenQASM is designed as a quantum assembly language for expressing circuits and control flow, a full-stack scheduler should eventually support a wider subset of the language, including repeated blocks, measurement operations, resets, classical conditions, and structured subroutines. This would allow the compiler to handle not only benchmark circuits, but also realistic error-correction programs. Before reaching that point, however, the current scheduler must first become more competitive with existing neutral-atom compilers such as PowerMOVE. Only then can we fairly evaluate whether unzoned shuttling, zoned shuttling, or a hybrid approach provides the best tradeoff between execution time, movement overhead, and fidelity.

A major limitation of the current implementation is the overhead caused by repeated loading and unloading between the SLM and AOD. As seen in the comparison with Enola and PowerMOVE, `Naive_N_Dag` often loses performance because it frequently returns atoms to the SLM and reloads them in later stages. This transfer overhead can dominate the benefits of parallel movement. A future implementation should preserve useful AOD groups across stages, even when the next stage is not identical to the current one. For example, if most atoms in the current AOD group are reused in the next stage, the solver should keep those atoms loaded and only unload or load the atoms that change. This would avoid unnecessary SLM-AOD transfers and would likely improve execution time substantially. Another important direction is improving the stage construction method. The current solver uses a DAG-based schedule that preserves dependencies but does not exploit all possible commutation structure in the circuit. When this project began, we assumed that the incoming circuit would already be optimized. However, even an optimized circuit can contain partially commuting gates that allow additional reordering without changing the logical operation. Future implementations could

use edge-coloring or related graph-theoretic methods to reduce the number of two-qubit interaction stages, similar to the strategies used by Enola and PowerMOVE. This optimization may be especially useful if the stage construction is designed around AOD movement groups rather than only around logical gate layers. The movement-grouping heuristic should also be extended. In the current `Naive_N_Dag` solver, the alignment threshold controls whether gates in the same DAG stage can be moved together. A more flexible version could reinterpret this score in terms of AOD occupancy and reuse. For example, an alignment value of -1 could allow all atoms currently loaded in the AOD to move together and only force unloading when two atoms inside the AOD must interact with one another. A value of 0 could allow only atoms with immediate interactions in the current stage to move together, while a value of 1 could preserve the current restrictive behavior where only highly parallel movements are grouped. This would give the user more direct control over the tradeoff between parallelism, unnecessary movement, and transfer overhead.

Fault-tolerant workloads also motivate a more hardware-aware choice of which qubits should move. In the current implementation, either qubit in a two-qubit gate may be selected as the moving atom. However, for error-correction circuits such as the $[[144, 12, 12]]$ bivariate bicycle code, it may be preferable to move ancilla qubits over data qubits. Data qubits must preserve logical information across many cycles, so moving them unnecessarily may introduce additional infidelity. Ancilla qubits, by contrast, are measured and reinitialized during syndrome extraction, making them more natural candidates for repeated shuttling. A future scheduler should therefore distinguish between data and ancilla qubits and bias movement decisions toward ancilla motion whenever possible. If both qubits are data qubits, the current general movement strategy could still be used. The routing heuristic itself also needs improvement. The current solver simplifies routing by using movement vectors and greedy local decisions. While this keeps compilation transparent, it does not globally optimize the movement schedule. Future work could incorporate more sophisticated routing methods, such as hypercube-based movement strategies similar to those used in atom-array routing work [7], or dynamic-programming heuristics that optimize stage transitions from the current atom positions. These methods would increase compilation complexity, but neutral-atom compilation is an offline classical optimization problem, so it may be reasonable to spend significantly more classical compute in order to reduce quantum execution time. The acceptable complexity depends on benchmark size, but even polynomially expensive approaches, such as methods scaling roughly as $O((gn)^4)$ for g gates and n qubits, may be worth exploring if they substantially improve final schedule quality. The physical AOD model is another simplification. The current implementation does not allow AOD stretching or expansion during execution. This restriction makes the solver easier to implement and analyze, but it limits the set of parallel movements that can be represented. Allowing the AOD rows and columns to stretch, contract, or otherwise adjust their spacing could make it possible to execute more gates in parallel and reduce unnecessary motion. Future versions of the scheduler should add configurable AOD stretching and compare its effect on execution time and movement fidelity. The initial placement method remains a major bottleneck. The simulated-annealing results show that placement quality has a large effect on execution time, especially for structured circuits such as the $[[144, 12, 12]]$ bivariate bicycle code. The current `fast_sa` cost function and greedy moves are not sufficient to reliably recover high-quality layouts. Future work should improve the placement cost function, add larger and more structured placement moves, and incorporate circuit-specific information such as repeated ancilla-data interactions. A clear validation target is the $[[144, 12, 12]]$ benchmark: an improved solver should reduce execution time both when it must initialize the lattice from scratch and when it is given a structured initial layout. Achieving this would provide strong evidence that unzoned shuttling can become competitive with PowerMOVE and other zone-based approaches.

A final direction is to add a separate solver for hardware that does not rely on conventional two-dimensional AOD motion. Recent work on rastering-based neutral-atom control proposes a device capable of moving qubits independently in arbitrary directions at approximately constant speed [16]. While 2D AODs are currently more common and widely used, this alternative control model removes one of the main constraints in our current scheduler: atoms no longer need to move as coupled rows and columns or share compatible movement directions. In principle, this could substantially reduce execution time and make schedules less sensitive to the initial placement. However, this additional freedom also introduces new scheduling challenges. If atoms can move independently, the solver must reason about congestion, crossing

trajectories, collision avoidance, and continuous path planning rather than only grouped AOD-compatible motion. One possible approach is to extend the highway abstraction into a two-lane routing model, where each highway contains two directed lanes so that atoms can move in opposite directions without blocking one another. This would preserve some of the structure that makes the current solver tractable while allowing more flexible routing than the 2D AOD model. Implementing such a solver would provide a useful comparison between conventional AOD-based shuttling and more general independent-motion architectures.

6 Acknowledgments

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Department of Energy Computational Science Graduate Fellowship under Award Number(s) De-SC0026073.

7 Availability of Data and Code

All code developed and used in this project is openly available in the public GitHub repository https://github.com/Gerwinlab/NeutralMotion. This repository contains the full implementation of the atom-scheduler, supporting scripts, configuration files, and all data required to reproduce the results presented in this report. No proprietary datasets were used. Feel free to add and support this open source project.

References

[1] N. Nottingham, M.A. Perlin, D. Shah, R. White, H. Bernien, F.T. Chong, J.M. Baker. Circuit decompositions and scheduling for neutral atom devices with limited local addressability (2024). URL https://arxiv.org/abs/2307.14996

[2] R.B.S. Tsai, H. Silvério, L. Henriet. Pulse-level scheduling of quantum circuits for neutral-atom devices (2022). URL https://arxiv.org/abs/2206.05144

[3] O. Savola, A. Paler. Atlas: Efficient atom rearrangement for defect-free neutral-atom quantum arrays under transport loss (2025). URL https://arxiv.org/abs/2511.16303

[4] W. Tian, W.J. Wee, A. Qu, B.J.M. Lim, P.R. Datla, V.P.W. Koh, H. Loh, Parallel assembly of arbitrary defect-free atom arrays with a multitweezer algorithm. Physical Review Applied **19**(3) (2023). https://doi.org/10.1103/physrevapplied.19.034048. URL http://dx.doi.org/10.1103/PhysRevApplied.19.034048

[5] S. Wang, W. Zhang, T. Zhang, S. Mei, Y. Wang, J. Hu, W. Chen, Accelerating the assembly of defect-free atomic arrays with maximum parallelisms. Physical Review Applied **19**(5) (2023). https://doi.org/10.1103/physrevapplied.19.054032. URL http://dx.doi.org/10.1103/PhysRevApplied.19.054032

[6] B. Cimring, R. El Sabeh, M. Bacvanski, S. Maaz, I. El Hajj, N. Nishimura, A.E. Mouawad, A. Cooper, Efficient algorithms to solve atom reconfiguration problems. i. redistribution-reconfiguration algorithm. Physical Review A **108**(2) (2023). https://doi.org/10.1103/physreva.108.023107. URL http://dx.doi.org/10.1103/PhysRevA.108.023107

[7] N. Constantinides, A. Fahimniya, D. Devulapalli, D. Bluvstein, M.J. Gullans, J.V. Porto, A.M. Childs, A.V. Gorshkov. Optimal routing protocols for reconfigurable atom arrays (2024). URL https://arxiv.org/abs/2411.05061

[8] D.B. Tan, D. Bluvstein, M.D. Lukin, J. Cong, Compiling quantum circuits for dynamically field-programmable neutral atoms array processors. Quantum **8**, 1281 (2024). https://doi.org/10.22331/q-2024-03-14-1281. URL http://dx.doi.org/10.22331/q-2024-03-14-1281

[9] D. Schoenberger, S. Hillmich, M. Brandl, R. Wille. Shuttling for scalable trapped-ion quantum computers (2024). URL https://arxiv.org/abs/2402.14065

[10] D. Schoenberger, R. Wille. Orchestrating multi-zone shuttling in trapped-ion quantum computers (2025). URL https://arxiv.org/abs/2505.07928

[11] D. Schoenberger, S. Hillmich, M. Brandl, R. Wille. Using boolean satisfiability for exact shuttling in trapped-ion quantum computers (2023). URL https://arxiv.org/abs/2311.03454

[12] D.B. Tan, W.H. Lin, J. Cong, in *Proceedings of the 30th Asia and South Pacific Design Automation Conference* (ACM, 2025), ASPDAC '25, p. 921–929. https://doi.org/10.1145/3658617.3697778. URL http://dx.doi.org/10.1145/3658617.3697778

[13] J. Ruan, X. Fang, H. Zhang, A. Li, T. Humble, Y. Ding. Powermove: Optimizing compilation for neutral atom quantum computers with zoned architecture (2024). URL https://arxiv.org/abs/2411.12263

[14] J. Viszlai, W. Yang, S.F. Lin, J. Liu, N. Nottingham, J.M. Baker, F.T. Chong, in *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2025), p. 688–699. https://doi.org/10.1109/qce65121.2025.00080. URL http://dx.doi.org/10.1109/QCE65121.2025.00080

[15] T.C. Chen, Y.W. Chang, Modern floorplanning based on b/sup */-tree and fast simulated annealing. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **25**(4), 637–650 (2006). https://doi.org/10.1109/TCAD.2006.870076

[16] E. Bytyqi, J. Sinclair, J. Ramette, V. Vuletić. Device for mhz-rate rastering of arbitrary 2d optical potentials (2026). URL https://arxiv.org/abs/2602.16025